

Python Radio 59: Steampunk Radio



[Simon Quellen Field](#)

Follow

6 min read

.

2 days ago

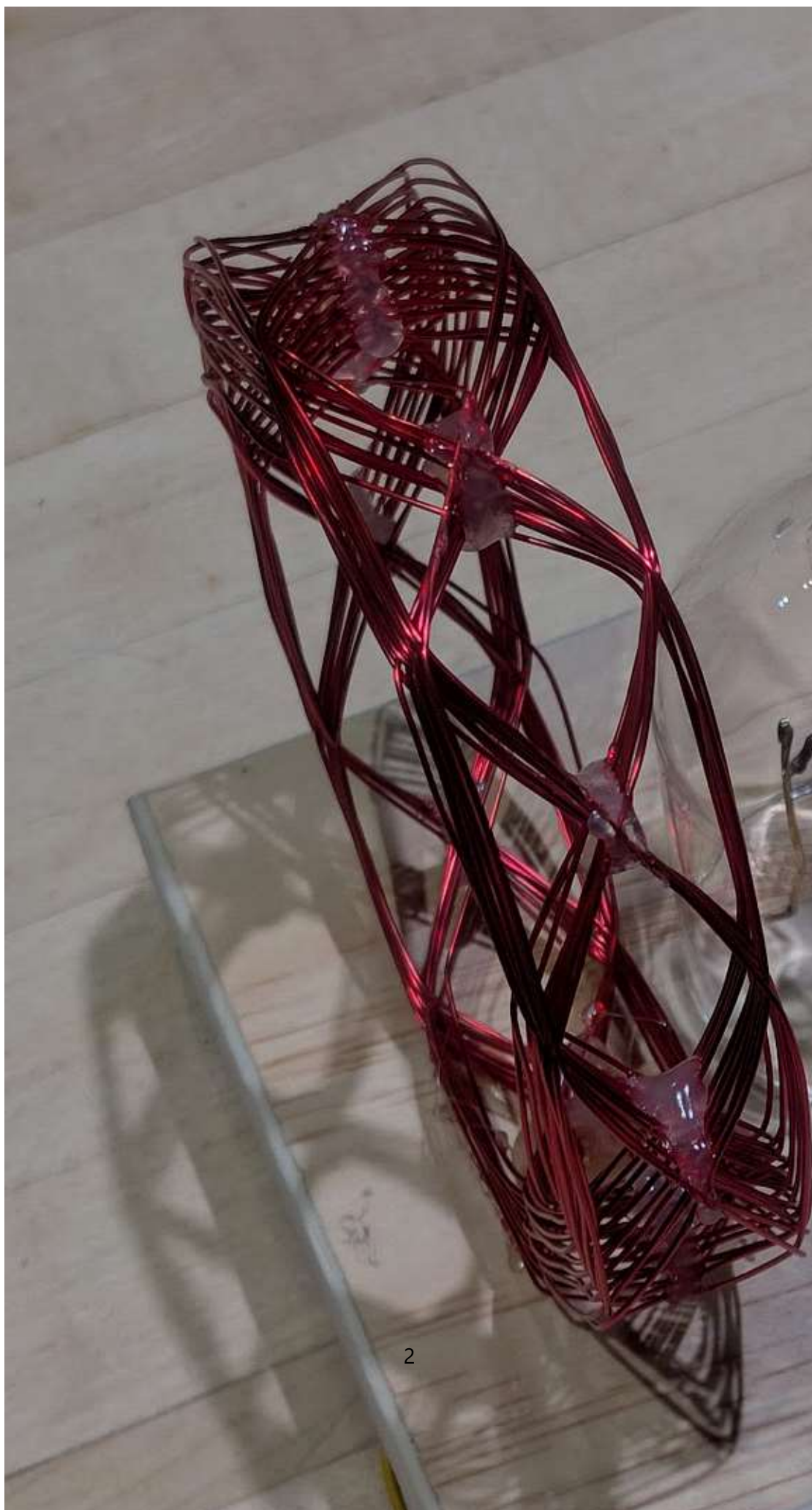
Listen

Share

More

We Finish Our Radio

Press enter or click to view image in full size



All photos by author

In the last few chapters of this series, we have built high-Q coils, homemade capacitors, and diodes. We have built Python programs to measure the Q of the tank circuit (the coil and the capacitor in parallel) and a program to chart the voltage vs current function of the diodes.

But we haven't measured the inductance and Q of the coil by itself. Let's remedy that oversight.

We can use the same general idea we used for the Q measurement. But we will replace the homemade capacitor with a precision capacitor with a capacitance guaranteed to be within 1% of its nominal value. I bought 10 100 pF 1% capacitors from Digikey for \$7.48.

With this capacitor, we should be able to tell what the inductance and Q of our coil is, using a little arithmetic:

$$F_R = \frac{1}{2\pi\sqrt{LC}}$$

The code looks like this:

```
import machine
```

```

import time
import math

# ===== USER CONFIGURATION =====
# --- Mode Selection ---
# Set to 'MEASURE_L' to analyze an inductor's L and Q.
# Set to 'MEASURE_C' to measure capacitance (Q reading will be less
meaningful).
MODE = 'MEASURE_L'

# --- Reference Component Value ---
# If MODE is 'MEASURE_L', enter your known HIGH-Q reference
capacitor value.
# Use a COG/NP0 or Film capacitor for best results.
# Example: 1 nF = 1.0e-9
REFERENCE_CAPACITOR_F = 0.1e-9

# If MODE is 'MEASURE_C', enter your reference inductor value.
# Example: 100 µH = 100.0e-6
REFERENCE_INDUCTOR_H = 100.0e-6

# --- Sweep Parameters ---
START_FREQ = 100_000 # 100 kHz
END_FREQ = 2000_000 # 2 MHz
NUM_STEPS = 200
SETTLE_TIME_MS = 10

# --- Hardware Pins ---
PWM_PIN_NUM = 16
ADC_PIN_NUM = 26
#
=====

# --- Pin Setup ---
pwm = machine.PWM(machine.Pin(PWM_PIN_NUM))
adc = machine.ADC(machine.Pin(ADC_PIN_NUM))
pwm.duty_u16(32768)

results = []

def interpolate_freq(target_val, freq1, val1, freq2, val2):
    """Linearly interpolates to find the frequency at a target
    value."""
    if (val2 - val1) == 0: return (freq1 + freq2) / 2
    return freq1 + (freq2 - freq1) * (target_val - val1) / (val2 -
val1)

def analyze_results(results):
    """Analyzes sweep data to find peak freq, bandwidth, and Q-
    factor."""
    if not results or len(results) < 3:
        return None, None, None, -1

    max_reading = -1
    max_idx = -1
    for i, (_, reading) in enumerate(results):
        if reading > max_reading:
            max_reading = reading

```

```

        max_idx = i

    if max_idx <= 0 or max_idx >= len(results) - 1 or max_reading
< 500:
        return results[max_idx][0] if max_idx != -1 else None,
None, None, max_reading

    resonant_freq = results[max_idx][0]
    half_power_reading = max_reading * 0.7071

    f_low, f_high = None, None
    for i in range(max_idx, 0, -1):
        if results[i][1] >= half_power_reading and results[i-1][1]
< half_power_reading:
            f_low = interpolate_freq(half_power_reading,
results[i-1][0], results[i-1][1], results[i][0], results[i][1])
            break

    for i in range(max_idx, len(results) - 1):
        if results[i][1] >= half_power_reading and results[i+1][1]
< half_power_reading:
            f_high = interpolate_freq(half_power_reading,
results[i][0], results[i][1], results[i+1][0], results[i+1][1])
            break

    if f_low is not None and f_high is not None:
        bandwidth = f_high - f_low
        if bandwidth > 0:
            q_factor = resonant_freq / bandwidth
            return resonant_freq, bandwidth, q_factor, max_reading

    return resonant_freq, None, None, max_reading

def main():
    print("=" * 40)
    if MODE == 'MEASURE_L':
        print(f"Mode: Inductor (L) Analyzer")
        print(f"Using reference C = {REFERENCE_CAPACITOR_F *
1e9:.2f} nF (ensure this is a high-Q cap)")
    elif MODE == 'MEASURE_C':
        print(f"Mode: Capacitance (C) Measurement")
        print(f"Using reference L = {REFERENCE_INDUCTOR_H *
1e6:.2f} µH")
    else:
        print("Error: Invalid MODE selected.")
        return
    print("=" * 40)

    log_start = math.log10(START_FREQ)
    log_end = math.log10(END_FREQ)
    log_step = (log_end - log_start) / (NUM_STEPS - 1)

    try:
        for i in range(NUM_STEPS):
            # ... (sweep loop as before) ...
            current_freq = 10**(log_start + i * log_step)
            pwm.freq(int(current_freq))
            time.sleep_ms(SETTLE_TIME_MS)

```

```

        reading_sum = 0
        for _ in range(50):
            reading_sum += adc.read_u16()
        adc_reading = reading_sum // 50
        results.append((current_freq, adc_reading))

        progress = int((i / (NUM_STEPS - 1)) * 30)
        print(f"[{'#' * progress}{ '-' * (30 - progress)}]
{current_freq/1000:.1f} kHz -> {adc_reading}", end='\r')

    print("\n" + "=" * 40)
    print("Sweep Complete! Analyzing...")

    resonant_freq, bandwidth, q_factor, peak_reading =
analyze_results(results)

    if resonant_freq is None or peak_reading < 500:
        print("No significant peak detected in the sweep
range.")
        print("Check wiring or adjust the START_FREQ/END_FREQ
range.")
    else:
        print(f"\nResonant Frequency (fr): {resonant_freq /
1000:.3f} kHz")

        # --- Calculate Component Value ---
        pi_squared = (2 * math.pi)**2
        if MODE == 'MEASURE_L':
            calculated_L = 1 / (pi_squared * resonant_freq**2 *
REFERENCE_CAPACITOR_F)
            print("-" * 35)
            print(f"Calculated Inductance (L): {calculated_L *
1e6:.2f} microHenrys")
            print("-" * 35)
        elif MODE == 'MEASURE_C':
            calculated_C = 1 / (pi_squared * resonant_freq**2 *
REFERENCE_INDUCTOR_H)
            print("-" * 35)
            print(f"Calculated Capacitance (C): {calculated_C *
1e12:.2f} pF")
            print("-" * 35)

        # --- Report Q-Factor ---
        if bandwidth is not None and q_factor is not None:
            print(f"Measured -3dB Bandwidth: {bandwidth /
1000:.3f} kHz")
            print(f"Measured Circuit Q-Factor: {q_factor:.2f}")
            if MODE == 'MEASURE_L':
                print("(This is a good approximation of the
INDUCTOR'S Q-factor)")
            else:
                print("(This value is dominated by the
reference inductor's Q-factor)")

        else:
            print("\nCould not determine Q-Factor.")
            print("The peak may be too broad or too close to
the sweep edges.")

```

```

except KeyboardInterrupt:
    print("\nSweep stopped by user.")
finally:
    pwm.deinit()
    print("PWM de-initialized.")

main()

```

Running the code on our RP2350 Raspberry Pi Pico 2, we get this output:

```

=====
Mode: Inductor (L) Analyzer
Using reference C = 0.10 nF (ensure this is a high-Q cap)
=====
[#####] 2000.0 kHz -> 11454
=====
Sweep Complete! Analyzing...

Resonant Frequency (fr): 590.833 kHz
-----
Calculated Inductance (L): 725.62 microHenrys
-----
Measured -3dB Bandwidth: 7.809 kHz
Measured Circuit Q-Factor: 75.66
(This is a good approximation of the INDUCTOR'S Q-factor)
PWM de-initialized.

```

To test the accuracy, let's run it again:

```

=====
Mode: Inductor (L) Analyzer
Using reference C = 0.10 nF (ensure this is a high-Q cap)
=====
[#####] 2000.0 kHz -> 11221
=====
Sweep Complete! Analyzing...

Resonant Frequency (fr): 590.833 kHz
-----
Calculated Inductance (L): 725.62 microHenrys
-----
Measured -3dB Bandwidth: 7.808 kHz
Measured Circuit Q-Factor: 75.67
(This is a good approximation of the INDUCTOR'S Q-factor)
PWM de-initialized.

```

The numbers agree within our 1% tolerance.

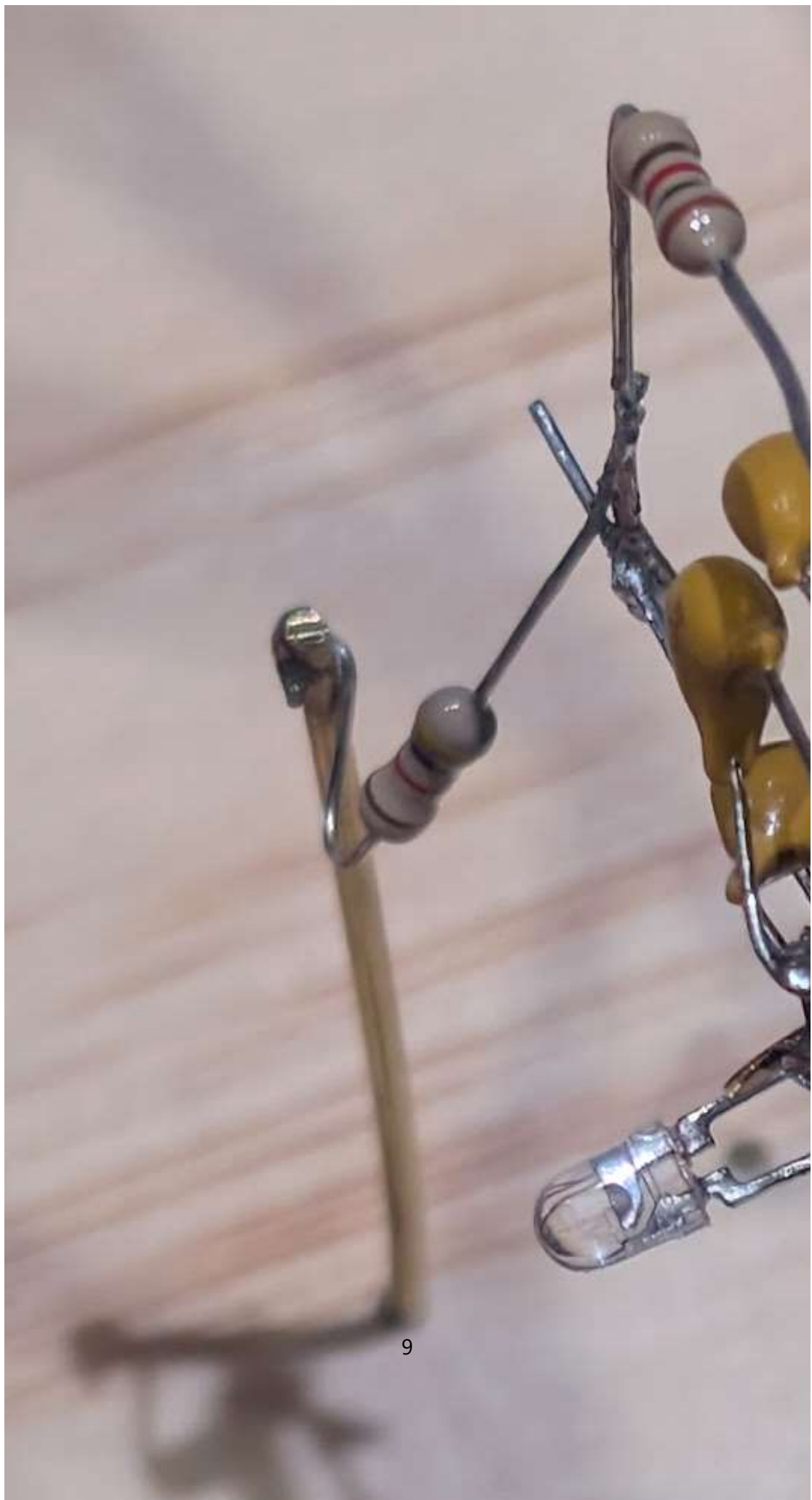
Now that we have a value for our coil, we can use it to test our

homemade capacitors and calculate their capacitance. Unfortunately, the Q of the coil will almost always be lower than the Q of the capacitor, so we won't be able to give a Q value for the capacitor.

A Tuned Radio Frequency Receiver

We have built several receivers in this series, but we have yet to build one of the simpler ones, a Tuned Radio Frequency (TRF) receiver.

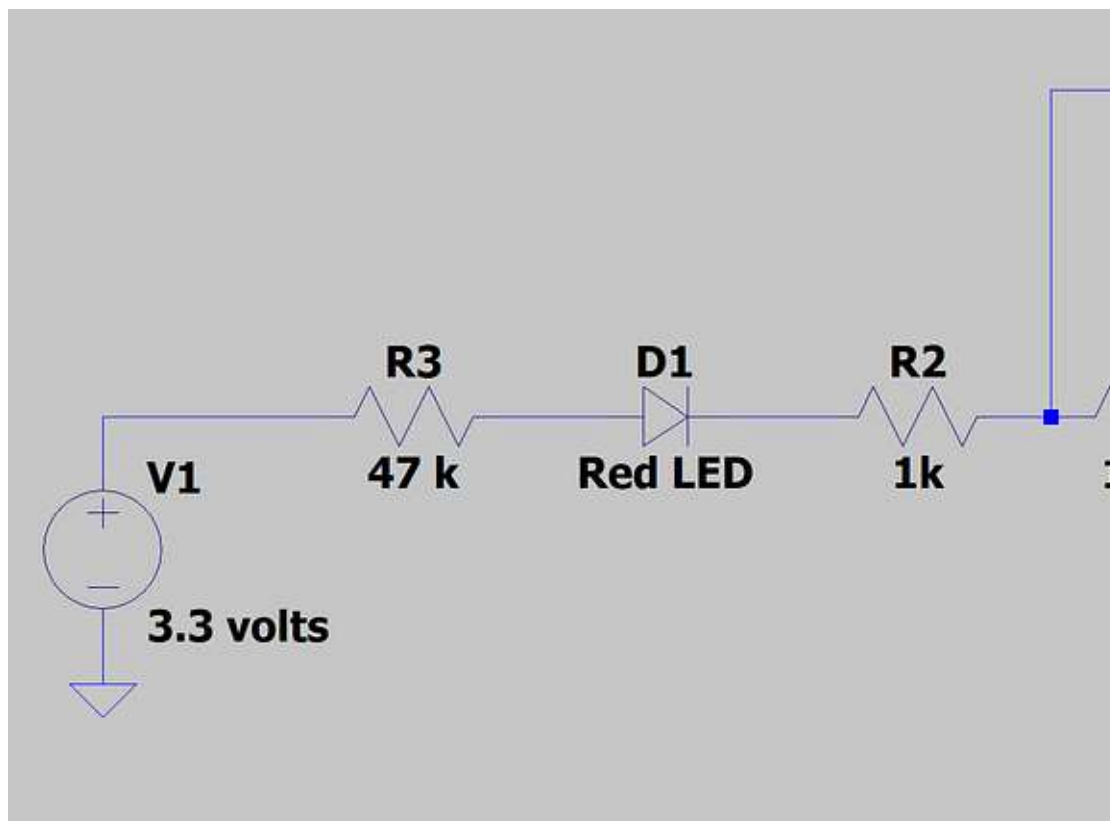
Since we have our homemade coil and our homemade capacitor, let's build a homemade TRF radio. We could use a printed circuit board or a breadboard, but let's go full steampunk and build the circuit in 3D:



We will use an MK484 TRF receiver integrated circuit as the core of our radio. It has ten transistors and will detect and amplify the AM signals received in our coil. The coil will be the antenna, as well as half of the tuning tank circuit.

Here is the schematic:

Press enter or click to view image in full size



I used a 3.3-volt lithium-ion 18650 battery to power the radio because, instead of the earphone the MK484 normally supports, I wanted to hide a small speaker and amplifier module under the board.

The MK484 uses a 1.5-volt battery, and 3.3 volts would destroy the chip (ask me how I know...). The red LED drops the 3.3 volts down to about 1.6 volts, which is under the 1.8 volt maximum in the MK484

spec sheet.

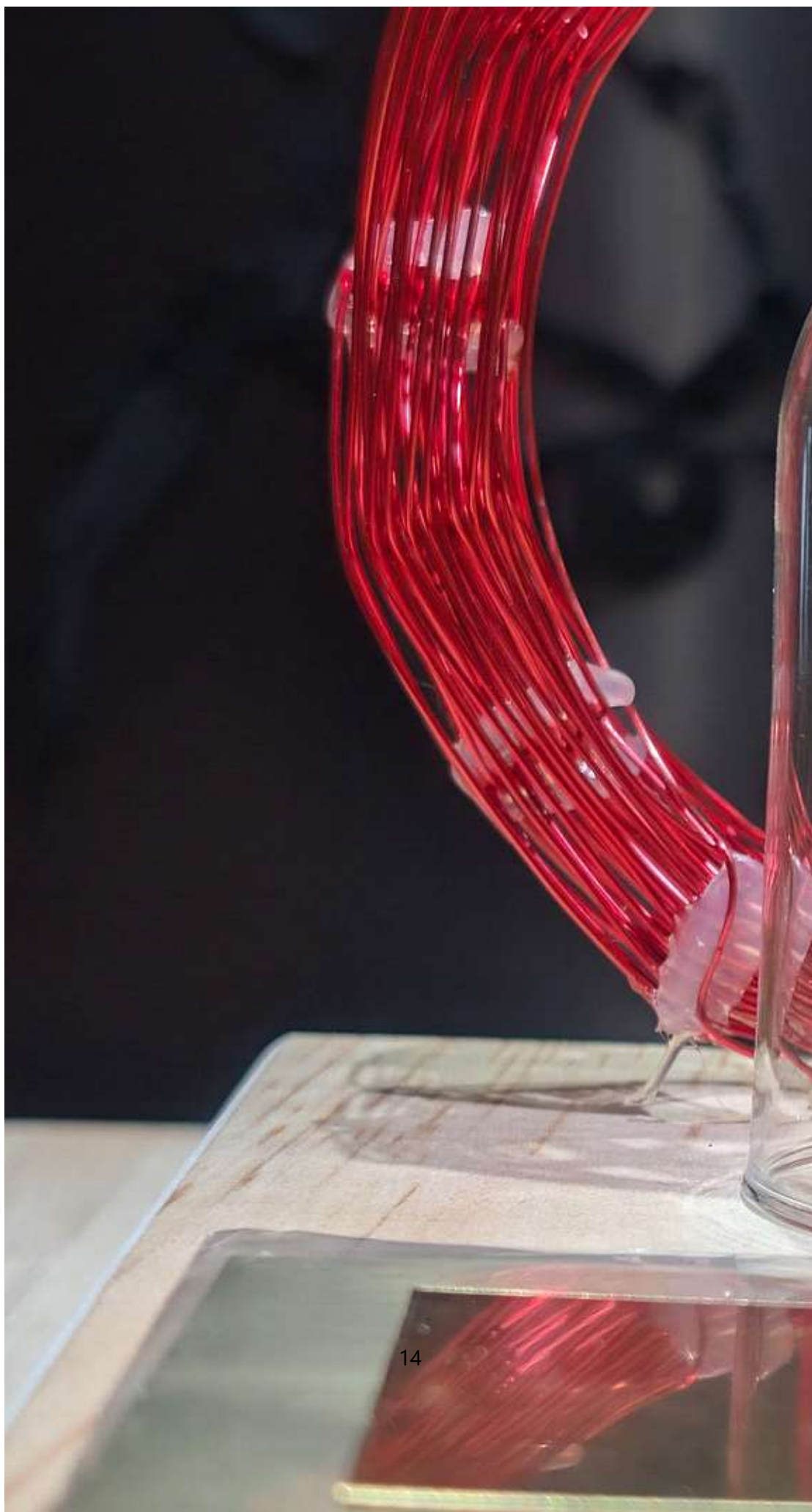
Press enter or click to view image in full size



The coil stands on edge, and the homemade variable capacitor lies flat. I made it of two brass sheets separated by a thin sheet of polypropylene. To tune the radio, you slide the top sheet partially off the bottom sheet.

The battery, speaker, and amplifier module hide underneath.

A nice little bell jar protects the circuit sculpture.
Press enter or click to view image in full size



And that is our little steampunk radio!
Press enter or click to view image in full size



Programming

Software Development

Python